

Yesterday 3:09 PM

java 8+ question
what is Lambda
What is a functional interface?
What is @FunctionalInterface?
Can a functional interface have multiple default methods?
Can it have static methods?
What is a lambda expression?
Lambda vs anonymous inner class.
What is effectively final?
Why must local variables used inside lambdas be
final/effectively final?



Absolutely. For a **5-year Java developer interview**, don't just memorize definitions. You should be able to explain **what it is, why it exists, how it works, and give a practical example**.

Java 8+ — Lambda & Functional Interface

1. What is Lambda?

Interview answer

A **lambda expression** is a concise way of representing an implementation of a **functional interface**.

It allows us to pass behavior as data, instead of creating a separate class or method implementation.

Syntax

```
</> Java
```



```
(parameters) -> expression
```

or

```
</> Java   
  
(parameters) -> {  
    // multiple statements  
}
```

Example

Without lambda:

```
</> Java   
  
Runnable r = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Running");  
    }  
};
```

With lambda:

```
</> Java   
  
Runnable r = () -> System.out.println("Running");
```

Here:

```
</> Java   
  
() -> System.out.println("Running")
```

is the lambda expression.

Another example

```
</> Java   
  
Comparator<Integer> comparator =  
    (a, b) -> Integer.compare(a, b);
```

And commonly with Streams:

</> Java



```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);

numbers.stream()
    .filter(n -> n % 2 == 0)
    .forEach(n -> System.out.println(n));
```

Here:

</> Java



```
n -> n % 2 == 0
```

represents the filtering behavior.

Important point

A lambda **doesn't have a type by itself**.

Its target type comes from a **functional interface**.

For example:

</> Java



```
Predicate<Integer> p = n -> n > 10;
```

The lambda:

</> Java



```
n -> n > 10
```

gets its meaning from:

</> Java



```
Predicate<Integer>
```

2. What is a Functional Interface?

A **functional interface** is an interface that has **exactly one abstract method**.

Example:

```
</> Java   
  
@FunctionalInterface  
interface Calculator {  
    int calculate(int a, int b);  
}
```

Now we can implement it using lambda:

```
</> Java   
  
Calculator calculator = (a, b) -> a + b;  
  
System.out.println(calculator.calculate(10, 20));
```

Output:

```
30 
```

Important interview point

A functional interface can have:

- exactly **one abstract method**
- multiple default methods
- multiple static methods
- methods inherited from `Object`

For example:

```
</> Java   
  
@FunctionalInterface  
interface MyInterface {  
  
    void execute();  
  
    default void start() {  
        System.out.println("Start");  
    }  
  
    default void stop() {  
        System.out.println("Stop");  
    }  
}
```

```
}

static void info() {
    System.out.println("Info");
}
}
```

This is still a functional interface because it has only:

```
</> Java

void execute();
```



as an abstract method.

3. What is @FunctionalInterface ?

`@FunctionalInterface` is a compiler annotation used to indicate that an interface is intended to be a functional interface.

Example:

```
</> Java

@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```



The compiler checks that the interface has only one abstract method.

What happens if we add another abstract method?

```
</> Java

@FunctionalInterface
interface Calculator {

    int add(int a, int b);

    int subtract(int a, int b);
}
```



Compilation error.

Because now there are two abstract methods.

Is `@FunctionalInterface` mandatory?

No.

This is valid:

 `</> Java`

```
interface Calculator {  
    int add(int a, int b);  
}
```

It is still a functional interface.

But `@FunctionalInterface` provides compiler validation and communicates developer intent.

Good interview statement

"`@FunctionalInterface` is not what makes an interface functional. The interface is functional because it has exactly one abstract method. The annotation allows the compiler to enforce that contract."

4. Can a Functional Interface have multiple default methods?

Yes.

Example:

 `</> Java`

```
@FunctionalInterface  
interface Payment {  
  
    void pay();  
  
    default void validate() {  
        System.out.println("Validating payment");  
    }  
}
```

```
default void log() {  
    System.out.println("Logging payment");  
}  
}
```

It is still a functional interface because there is only one **abstract** method:

</> Java



```
void pay();
```

Default methods already have an implementation, so they don't count toward the single abstract method requirement.

5. Can a Functional Interface have static methods?

Yes.

Example:

</> Java



```
@FunctionalInterface  
interface Payment {  
  
    void pay();  
  
    static void printInfo() {  
        System.out.println("Payment interface");  
    }  
}
```

You call the static method using the interface:

</> Java



```
Payment.printInfo();
```

You **cannot** call it through the implementation object:

</> Java



```
Payment p = () -> System.out.println("Pay");
```

```
// p.printInfo(); // ❌
```

Static interface methods belong to the interface itself.

6. What is a Lambda Expression?

This is slightly different from asking "What is Lambda?"

A **lambda expression** is an anonymous function-like expression that provides the implementation of the single abstract method of a functional interface.

Example:

```
</> Java  
  
(a, b) -> a + b
```



Suppose:

```
</> Java  
  
@FunctionalInterface  
interface Calculator {  
    int calculate(int a, int b);  
}
```



Then:

```
</> Java  
  
Calculator c = (a, b) -> a + b;
```



Conceptually:

```
Functional Interface  
  |  
  | provides target type  
  ↓  
(a, b) -> a + b  
  |  
  ↓  
implementation of calculate()
```



You can invoke it:

</> Java

```
System.out.println(c.calculate(10, 20));
```

Output:

30

7. Lambda vs Anonymous Inner Class

This is a very common interview question.

Anonymous class

</> Java

```
Runnable r = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Running");  
    }  
};
```

Lambda

</> Java

```
Runnable r = () -> System.out.println("Running");
```

Main differences

Feature	Lambda	Anonymous Inner Class
Introduced	Java 8	Older Java
Syntax	Concise	Verbose
Works with	Functional interfaces	Interfaces/classes

Feature	Lambda	Anonymous Inner Class
Can have multiple methods	No, lambda represents one SAM	Yes
this	Refers to enclosing object	Refers to anonymous object
Can declare fields	No	Yes
Can have constructors	No	Yes
Readability	Usually better	More verbose

Most important difference: this

This is a good 5-year interview point.

Consider:

</> Java



```
class Test {  
  
    void execute() {  
  
        Runnable lambda = () -> {  
            System.out.println(this);  
        };  
  
        Runnable anonymous = new Runnable() {  
            @Override  
            public void run() {  
                System.out.println(this);  
            }  
        };  
    }  
}
```

Inside the lambda:

</> Java



```
this
```

refers to the **enclosing** `Test` **object**.

Inside the anonymous class:

```
</> Java
```



```
this
```

refers to the **anonymous inner-class object**.

That's an important behavioral difference.

8. What is Effectively Final?

This is extremely important when discussing lambdas.

A local variable is **effectively final** if you assign a value to it once and never modify it afterward.

Example:

```
</> Java
```



```
int x = 10;

Runnable r = () -> {
    System.out.println(x);
};
```

We didn't explicitly write:

```
</> Java
```



```
final int x = 10;
```

But `x` is **effectively final** because we never change it.

Therefore this is valid.

Explicitly final

```
</> Java

final int x = 10;

Runnable r = () -> {
    System.out.println(x);
};
```



Effectively final

```
</> Java

int x = 10;

Runnable r = () -> {
    System.out.println(x);
};
```



Both work.

Not effectively final

```
</> Java

int x = 10;

x = 20;

Runnable r = () -> {
    System.out.println(x);
};
```



Compilation error.

Because `x` was modified.

9. Why must local variables used inside lambdas be final/effectively final?

This is where interviewers often expect more than:

"Because Java says so."

The important concept is **variable lifetime and captured values**.

Consider:

```
</> Java   
  
void test() {  
  
    int x = 10;  
  
    Runnable r = () -> {  
        System.out.println(x);  
    };  
  
    r.run();  
}
```

The lambda is capturing the local variable `x`.

But `x` is a **local variable stored in the method's execution context/stack frame**.

The lambda object can potentially live longer than the method invocation.

Conceptually:

```
test()   
|  
|-- local variable x = 10  
|  
|-- lambda captures x  
|  
└─ method returns  
    |  
    └─ lambda may still exist
```

Java therefore captures the **value** of the local variable rather than allowing the lambda to hold a mutable reference to the stack local variable.

Requiring the variable to be final/effectively final makes this behavior predictable.

Example

</> Java



```
int x = 10;
```

```
Runnable r = () -> System.out.println(x);
```

```
x = 20; // ❌
```

If this were allowed, there would be ambiguity about what the lambda should observe.

Should it print:

10



or

20



depending on when it runs?

Java avoids this by requiring captured local variables to be final/effectively final.

Important nuance: Objects can still be mutated

This is a **very good** interview trap.

Consider:

</> Java



```
List<String> names = new ArrayList<>();
```

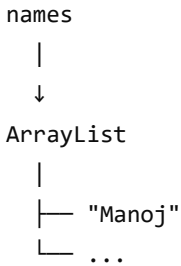
```
Runnable r = () -> {  
    names.add("Manoj");  
};
```

This is allowed.

Why?

Because the **reference** `names` isn't changed.

But the object that `names` refers to can be modified.



This would NOT be allowed:

```

</> Java

List<String> names = new ArrayList<>();

Runnable r = () -> {
    System.out.println(names);
};

names = new ArrayList<>(); // ❌
  
```

Because now the reference itself has been reassigned.

Remember:

Effectively final applies to the variable/reference, not necessarily to the object being referenced.

★ Interview-Level Summary

If an interviewer asks these questions rapidly, you can answer like this:

Lambda

Lambda is a concise way to provide the implementation of a functional interface's single abstract method. It allows us to pass behavior as an argument and is heavily used with Streams and functional programming.

Functional Interface

A functional interface has exactly one abstract method. It can still have multiple default and static methods. Examples include `Runnable`, `Predicate`, `Function`, `Consumer` and

Supplier.

@FunctionalInterface

It is a compiler-level annotation used to enforce the functional-interface contract. It isn't mandatory.

Multiple default methods?

Yes. Default methods have implementations and therefore don't count as abstract methods.

Static methods?

Yes. A functional interface can have multiple static methods. They belong to the interface and are called using the interface name.

Lambda vs Anonymous class

Lambda is more concise and works with functional interfaces. An anonymous class can implement an interface or extend a class and can have its own fields and methods. Another key difference is that `this` in a lambda refers to the enclosing object, whereas `this` in an anonymous class refers to the anonymous object.

Effectively final

A local variable is effectively final when it is assigned once and never reassigned. It doesn't need the `final` keyword.

Why final/effectively final?

Lambdas can capture local variables, and those local variables may outlive the method invocation. Java captures their value rather than providing mutable access to the local variable, so Java requires captured local variables to be final or effectively final.



Here is the **quick interview-ready syntax** for each Java 8 functional interface:

1. **Predicate<T>** → **takes 1 input, returns boolean**

</> Java



```
Predicate<Integer> p = n -> n > 10;  
System.out.println(p.test(20)); // true
```

2. **Function<T, R>** → **takes 1 input, returns 1 result**

</> Java



```
Function<String, Integer> f = s -> s.length();  
System.out.println(f.apply("Java")); // 4
```

3. **Consumer<T>** → **takes 1 input, returns nothing**

</> Java



```
Consumer<String> c = s -> System.out.println(s);  
c.accept("Hello"); // Hello
```

4. **Supplier<T>** → **takes no input, returns a value**

</> Java



```
Supplier<Double> s = () -> Math.random();  
System.out.println(s.get());
```

5. **UnaryOperator<T>** → **takes 1 input, returns same type**

</> Java



```
UnaryOperator<Integer> op = n -> n * 2;  
System.out.println(op.apply(10)); // 20
```

6. BinaryOperator<T> → takes 2 inputs of same type, returns same type

</> Java



```
BinaryOperator<Integer> op = (a, b) -> a + b;
System.out.println(op.apply(10, 20)); // 30
```

7. BiFunction<T, U, R> → takes 2 inputs, returns a result

</> Java



```
BiFunction<Integer, Integer, Integer> f = (a, b) -> a + b;
System.out.println(f.apply(10, 20)); // 30
```

8. BiPredicate<T, U> → takes 2 inputs, returns boolean

</> Java



```
BiPredicate<String, Integer> p = (s, n) -> s.length() > n;
System.out.println(p.test("Java", 3)); // true
```

9. BiConsumer<T, U> → takes 2 inputs, returns nothing

</> Java



```
BiConsumer<String, Integer> c = (name, age) ->
    System.out.println(name + " " + age);
c.accept("Manoj", 25);
```

Easy way to remember

Interface	Input	Output	Method
Predicate<T>	1	boolean	test()
Function<T,R>	1	result	apply()
Consumer<T>	1	nothing	accept()
Supplier<T>	0	result	get()
UnaryOperator<T>	1	same type	apply()

Interface	Input	Output	Method
BinaryOperator<T>	2	same type	apply()
BiFunction<T,U,R>	2	result	apply()
BiPredicate<T,U>	2	boolean	test()
BiConsumer<T,U>	2	nothing	accept()

Shortcut:

Predicate = check → Function = transform → Consumer = consume → Supplier = provide



Difference between Function and Consumer.
 Difference between Supplier and Function.
 When would you use Predicate?
 What is Function.andThen()?
 What is Function.compose()?
 Difference between UnaryOperator and Function.
 Write your own functional interface.
 Give a real production use case for Supplier.

For a 5-year Java interview, answer these with both the concept and a small example.

1. Difference between Function and Consumer

Function<T, R>

Takes an input and **returns a result**.

</> Java



```
Function<String, Integer> f = s -> s.length();
```

```
Integer result = f.apply("Java"); // 4
```

Consumer<T>

Takes an input but **does not return anything**.

```
</> Java

Consumer<String> c = s -> System.out.println(s);

c.accept("Java");
```

Interview line

Function transforms data and returns a result, whereas Consumer performs an action on data without returning a result.

2. Difference between Supplier and Function

Supplier<T>

Takes **no input** and produces a value.

```
</> Java

Supplier<String> s = () -> "Hello";

String result = s.get();
```

Function<T,R>

Takes **one input** and produces a result.

```
</> Java

Function<String, Integer> f = str -> str.length();

Integer result = f.apply("Hello");
```

	Supplier	Function
Input	0	1

	Supplier	Function
Output	1	1
Method	get()	apply()

Interview line

Supplier generates/provides a value without requiring input, while Function transforms an input into an output.

3. When would you use Predicate ?

Use `Predicate<T>` when you need to **test a condition** and get `true/false`.

</> Java



```
Predicate<Integer> isEven = n -> n % 2 == 0;

System.out.println(isEven.test(10)); // true
```

Common production use cases

Filtering data:

</> Java



```
List<Integer> result = numbers.stream()
    .filter(n -> n > 100)
    .toList();
```

The condition:

</> Java



```
n -> n > 100
```

is essentially a `Predicate`.

Other examples

- Validate input

- Filter records
- Authorization checks
- Business rules
- Conditional processing

Interview line

I use Predicate when I need to represent a reusable boolean condition, especially for filtering, validation, or business-rule evaluation.

4. What is `Function.andThen()` ?

`andThen()` allows you to **execute one Function and then pass its result to another Function**.

Suppose:

`</>` Java



```
Function<Integer, Integer> multiply =  
    x -> x * 2;  
  
Function<Integer, Integer> add =  
    x -> x + 10;
```

Using `andThen()` :

`</>` Java



```
Function<Integer, Integer> result =  
    multiply.andThen(add);  
  
System.out.println(result.apply(5));
```

Execution:

5

↓

multiply → 10

↓

add → 20



So:

```
multiply.andThen(add)
```



means:

```
add(multiply(x))
```



Easy interview line

`andThen()` executes the current function first and then executes the next function.

5. What is `Function.compose()` ?

`compose()` does the **opposite order**.

`</>` Java



```
Function<Integer, Integer> multiply =  
    x -> x * 2;  
  
Function<Integer, Integer> add =  
    x -> x + 10;  
  
Function<Integer, Integer> result =  
    multiply.compose(add);  
  
System.out.println(result.apply(5));
```

Execution:

```
5  
↓  
add    → 15  
↓  
multiply → 30
```



So:

```
multiply.compose(add)
```



means:

```
multiply(add(x))
```



Remember this

andThen:

$A \rightarrow B$

compose:

$B \rightarrow A$



Or:

andThen = first me, then you

compose = first you, then me

6. Difference between UnaryOperator and Function

Both take **one** argument and return one result.

The difference is the **types**.

Function

</> Java

```
Function<String, Integer> f =  
    s -> s.length();
```



Input:

```
String
```



Output:

```
Integer
```



So:

$T \rightarrow R$ 

UnaryOperator

Input and output must be the **same type**.

</> Java



```
UnaryOperator<Integer> op =  
    n -> n * 2;
```

 $\text{Integer} \rightarrow \text{Integer}$ 

Interview answer

`UnaryOperator<T>` is a specialized `Function<T,T>` where the input and output types are the same.

In fact, conceptually:

</> Java



```
UnaryOperator<Integer>
```

is equivalent to:

</> Java



```
Function<Integer, Integer>
```

7. Write your own Functional Interface

Suppose we want to create an interface for calculating discounts.

</> Java



```
@FunctionalInterface  
interface DiscountCalculator {
```

```
double calculate(double price);  
}
```

Use it with lambda:

```
</> Java   
  
DiscountCalculator discount =  
    price -> price * 0.10;  
  
System.out.println(discount.calculate(1000));
```

Output:

```
100.0 
```

You can also use it in a service:

```
</> Java   
  
class OrderService {  
  
    double calculateFinalPrice(  
        double price,  
        DiscountCalculator calculator) {  
  
        return price - calculator.calculate(price);  
    }  
}
```

Then:

```
</> Java   
  
DiscountCalculator discount =  
    price -> price * 0.10;  
  
OrderService service = new OrderService();  
  
double finalPrice =  
    service.calculateFinalPrice(1000, discount);
```

Why is this useful?

Because the business logic can be passed as behavior:

OrderService



DiscountCalculator



Regular customer → 10%

Premium customer → 20%

Festival → 30%

You don't have to create separate classes for every small behavior.

8. Real production use case for Supplier

A very common use case is lazy creation of an expensive/default value.

For example, suppose a service needs a fallback configuration:

</> Java

```
Supplier<Config> configSupplier =
    () -> loadConfigFromDatabase();
```

You don't load the configuration until you actually need it:

</> Java

```
Config config = configSupplier.get();
```

Another very common example: `Optional.orElseGet()`

</> Java

```
User user = userRepository.findById(id)
    .orElseGet(() -> createDefaultUser());
```

Here:

</> Java

```
() -> createDefaultUser()
```

is a `Supplier<User>` .

The important advantage is **lazy evaluation**.

Compare:

</> Java



```
.orElse(createDefaultUser());
```

with:

</> Java



```
.orElseGet(() -> createDefaultUser());
```

`orElse()` evaluates its argument immediately, while `orElseGet()` calls the supplier only when the `Optional` is empty.

Production examples of Supplier

- Lazy initialization
- Fallback/default object creation
- Generating IDs/tokens
- Loading configuration when needed
- Expensive object creation
- Cache-miss value generation

Strong 5-year interview answer

"A production use case for Supplier is lazy evaluation. For example, in `Optional.orElseGet()`, I can provide a Supplier that creates a fallback object only when the value is actually absent. This avoids unnecessary computation or database calls."

★ One-page interview cheat sheet

Predicate<T>



Input: 1

Output: boolean

Method: `test()`

Use: condition/filtering

Function<T,R>

Input: 1

Output: R

Method: apply()

Use: transformation

Consumer<T>

Input: 1

Output: nothing

Method: accept()

Use: action/side effect

Supplier<T>

Input: 0

Output: T

Method: get()

Use: lazy value generation

UnaryOperator<T>

Input: T

Output: T

Method: apply()

Use: transform same type

BiFunction<T,U,R>

Input: 2

Output: R

Method: apply()

BiPredicate<T,U>

Input: 2

Output: boolean

Method: test()

BiConsumer<T,U>

Input: 2

Output: nothing

Method: accept()

And remember these two:

Function.andThen(B)

→ A first, B second

Function.compose(B)

→ B first, A second



And:

```
UnaryOperator<T>  
    =  
Function<T, T>
```



Why was java.time introduced?

LocalDate vs LocalDateTime.

Instant vs LocalDateTime.

ZonedDateTime.

OffsetDateTime.

Duration vs Period.

How would you store timestamps in a database?

How would you handle users from different time zones?

How do you convert UTC to IST?

Why is SimpleDateFormat problematic in multithreaded applications?

For a **5-year Java interview**, these are important because interviewers usually want to know not just the API, but **which date/time type you would choose in a real application**.

1. Why was java.time introduced?

Before Java 8, Java mainly used:

- java.util.Date
- java.util.Calendar
- SimpleDateFormat

They had several problems:

- APIs were confusing
- Date didn't clearly represent date vs time vs timezone
- Most classes were mutable
- SimpleDateFormat was not thread-safe
- Time-zone handling was cumbersome

Java 8 introduced the **java.time package**, based on the Joda-Time design.

Important characteristics:

- Immutable
- Thread-safe
- Clear API
- Better timezone support
- Separate classes for different concepts

Examples:

</> **Java**



```
LocalDate  
LocalDateTime  
Instant  
ZonedDateTime  
OffsetDateTime  
Duration  
Period
```

Interview answer

`java.time` was introduced in Java 8 to provide a modern, immutable, thread-safe and much clearer API for handling dates, times, time zones and durations.

2. LocalDate VS LocalDateTime

`LocalDate`

Represents **date only**.

</> **Java**



```
LocalDate date = LocalDate.now();  
  
System.out.println(date);
```

Example:

2026-09-01



Use it for:

- Birthday
- Holiday
- Due date
- Date of joining

You don't need a timezone.

LocalDateTime

Represents **date + time**, but **no timezone/offset**.

 **Java** 

```
LocalDateTime dateTime = LocalDateTime.now();  
  
System.out.println(dateTime);
```

Example:

2026-09-01T15:30:45 

Use it when timezone isn't relevant.

Interview shortcut

LocalDate 

↓

Date only

LocalDateTime

↓

Date + Time

↓

NO timezone

3. Instant VS LocalDateTime

This is a **very important** interview question.

Instant

Represents an **absolute point on the UTC timeline**.

</> Java



```
Instant now = Instant.now();  
  
System.out.println(now);
```

Example:

2026-09-01T10:00:00Z



Think:

"Exactly when did this event happen globally?"

Good for:

- Database timestamps
- Logs
- Event creation time
- API timestamps
- Audit fields

LocalDateTime

Has no timezone information.

</> Java



```
LocalDateTime dt =  
    LocalDateTime.of(2026, 9, 1, 15, 30);
```

It simply means:

September 1, 2026 at 3:30 PM

But **where?**

India?

London?

New York?

We don't know.

Key difference

`Instant`

= absolute moment on timeline



`LocalDateTime`

= date + time without timezone

Strong interview answer

I use `Instant` when I need to represent an absolute point in time, especially for persistence and distributed systems. I use `LocalDateTime` when the timezone is intentionally irrelevant.

4. What is `ZonedDateTime` ?

`ZonedDateTime` represents:

Date + Time + Time Zone



Example:

`</>` **Java**



```
ZonedDateTime time =  
    ZonedDateTime.now(  
        ZoneId.of("Asia/Kolkata")  
    );  
  
System.out.println(time);
```

It might look like:

2026-09-01T15:30+05:30[Asia/Kolkata]



Use case

Suppose your application has users in:

India
US
UK
Japan



You may need to represent:

10 AM in New York

Then timezone information matters.

</> Java



```
ZonedDateTime nyTime =  
    ZonedDateTime.now(  
        ZoneId.of("America/New_York")  
    );
```

Interview line

`ZonedDateTime` represents date, time and a full time-zone ID such as `Asia/Kolkata` or `America/New_York`, making it useful when timezone rules matter.

5. What is `OffsetDateTime` ?

`OffsetDateTime` represents:

Date + Time + UTC Offset



Example:

</> Java



```
OffsetDateTime time =  
    OffsetDateTime.now();  
  
System.out.println(time);
```

Example:

```
2026-09-01T15:30:00+05:30
```



Here:

```
+05:30
```



is the offset.

Difference from `ZonedDateTime`

`OffsetDateTime` :

```
2026-09-01T15:30+05:30
```



It knows the **offset**.

`ZonedDateTime` :

```
2026-09-01T15:30+05:30[Asia/Kolkata]
```



It knows the **actual timezone** and its rules.

Interview shortcut

`OffsetDateTime`

= Date + Time + Offset

`ZonedDateTime`

= Date + Time + Zone



6. Duration VS Period

Another common interview question.

Duration

Used for **time-based amounts**.

Think:

seconds
minutes
hours



Example:

</> Java



```
Duration duration =  
    Duration.between(start, end);  
  
System.out.println(duration.toMinutes());
```

Example:

2 hours 30 minutes



Period

Used for **date-based amounts**.

Think:

years
months
days



Example:

</> Java



```
Period period =  
    Period.between(  
        LocalDate.of(2025, 1, 1),  
        LocalDate.of(2026, 3, 10)  
    );  
  
System.out.println(period);
```

Could give:

P1Y2M9D



Easy memory trick

Duration → Time

Period → Date



Or:

Duration → hours/minutes/seconds

Period → years/months/days



7. How would you store timestamps in a database?

For **distributed/backend** systems, my preferred approach is:

Store timestamps in UTC.

For example:

</> Java

```
Instant createdAt = Instant.now();
```



Database:

2026-09-01 10:00:00 UTC



Then when returning the data to a user, convert it to their timezone.

Architecture

User

|

| India

↓

Application

|

| convert/store UTC

↓

Database



```
|  
| UTC  
↓  
Application  
|  
| convert to user's timezone  
↓  
User
```

Why UTC?

Because different servers/users can operate in different time zones.

If you store local times:

```
Server A → India  
Server B → US  
Server C → UK
```



you can easily create ambiguity.

Example

Store:

```
</> Java  
  
Instant createdAt = Instant.now();
```



Retrieve it:

```
</> Java  
  
Instant timestamp = ...;
```



Convert for the user:

```
</> Java  
  
ZonedDateTime userTime =  
    timestamp.atZone(  
        ZoneId.of("Asia/Kolkata")  
    );
```



Interview answer

For event timestamps such as `createdAt` , `updatedAt` , payment time or audit time, I would store UTC, preferably represented by `Instant` , and convert it to the user's timezone only at the presentation/API boundary.

8. How would you handle users from different time zones?

Don't store only:

```
10:00 AM
```



because we don't know which timezone.

Instead, maintain the user's timezone:

```
userId
timezone
-----
101
Asia/Kolkata

102
America/New_York
```



Then:

Step 1 — Store events in UTC

```
</> Java
```

```
Instant eventTime = Instant.now();
```



Step 2 — Store user's timezone

```
Asia/Kolkata
```



Step 3 — Convert when displaying

```
</> Java
```



```
ZonedDateTime userTime =  
    eventTime.atZone(  
        ZoneId.of("Asia/Kolkata")  
    );
```

Real-world example

Suppose database stores:

10:00 UTC



Indian user sees:

15:30 IST



New York user sees a different local time.

Same instant, different representation.

9. How do you convert UTC to IST?

IST is UTC+05:30.

Modern Java:

```
</> Java  
  
Instant utc = Instant.now();  
  
ZonedDateTime ist =  
    utc.atZone(  
        ZoneId.of("Asia/Kolkata")  
    );  
  
System.out.println(ist);
```



Or with a known UTC time:

```
</> Java  
  
Instant instant =  
    Instant.parse("2026-09-01T10:00:00Z");
```



```
ZonedDateTime ist =  
    instant.atZone(  
        ZoneId.of("Asia/Kolkata")  
    );  
  
System.out.println(ist);
```

Result:

```
2026-09-01T15:30+05:30[Asia/Kolkata]
```



Important

Prefer:

```
</> Java
```



```
ZoneId.of("Asia/Kolkata")
```

rather than manually adding 5 hours 30 minutes .

Why?

Because timezone APIs understand timezone rules, whereas manual arithmetic is fragile.

10. Why is SimpleDateFormat problematic in multithreaded applications?

SimpleDateFormat is **not thread-safe**.

Example:

```
</> Java
```



```
SimpleDateFormat format =  
    new SimpleDateFormat("yyyy-MM-dd");
```

If multiple threads share the same instance:

```
Thread 1 ─┐  
           └─> same SimpleDateFormat  
Thread 2 ─┐
```



Thread 3

they can interfere with its mutable internal state.

This can result in:

- Incorrect parsing
- Incorrect formatting
- Unexpected results
- Race conditions

Bad

</> Java



```
class DateUtil {  
  
    private static final SimpleDateFormat FORMAT =  
        new SimpleDateFormat("yyyy-MM-dd");  
  
    static String format(Date date) {  
        return FORMAT.format(date);  
    }  
}
```

If multiple threads call this concurrently, it's unsafe.

How did people solve it?

One approach was creating a new instance per call:

</> Java



```
SimpleDateFormat format =  
    new SimpleDateFormat("yyyy-MM-dd");
```

Another was synchronization:

</> Java



```
synchronized (format) {  
    format.format(date);  
}
```

```
}
```

But Java 8 provides a much better solution.

Use:

```
</> Java
```



```
DateTimeFormatter formatter =  
    DateTimeFormatter.ofPattern("yyyy-MM-dd");
```

`DateTimeFormatter` is **immutable and thread-safe**.

```
</> Java
```



```
LocalDate date = LocalDate.now();  
  
String result =  
    date.format(formatter);
```

Strong interview answer

`SimpleDateFormat` is mutable and not thread-safe, so sharing one instance across multiple threads can cause race conditions and incorrect results. In Java 8+, I prefer the immutable and thread-safe `DateTimeFormatter`.

★ Final Interview Cheat Sheet

Class	Meaning	Typical use
<code>LocalDate</code>	Date only	Birthday, holiday
<code>LocalDateTime</code>	Date + time, no zone	Local business date/time
<code>Instant</code>	Absolute UTC moment	DB timestamps, events, logs
<code>ZonedDateTime</code>	Date + time + timezone	User timezone operations
<code>OffsetDateTime</code>	Date + time + offset	APIs/protocols
<code>Duration</code>	Time-based amount	Hours/minutes/seconds

Class	Meaning	Typical use
Period	Date-based amount	Years/months/days

The most important rule for backend interviews:



And:

SimpleDateFormat	✗ mutable, not thread-safe	
DateTimeFormatter	✓ immutable, thread-safe	



ChatGPT can make mistakes. Check important info.